

Lab Assignment -1

Name : G.Abhilash

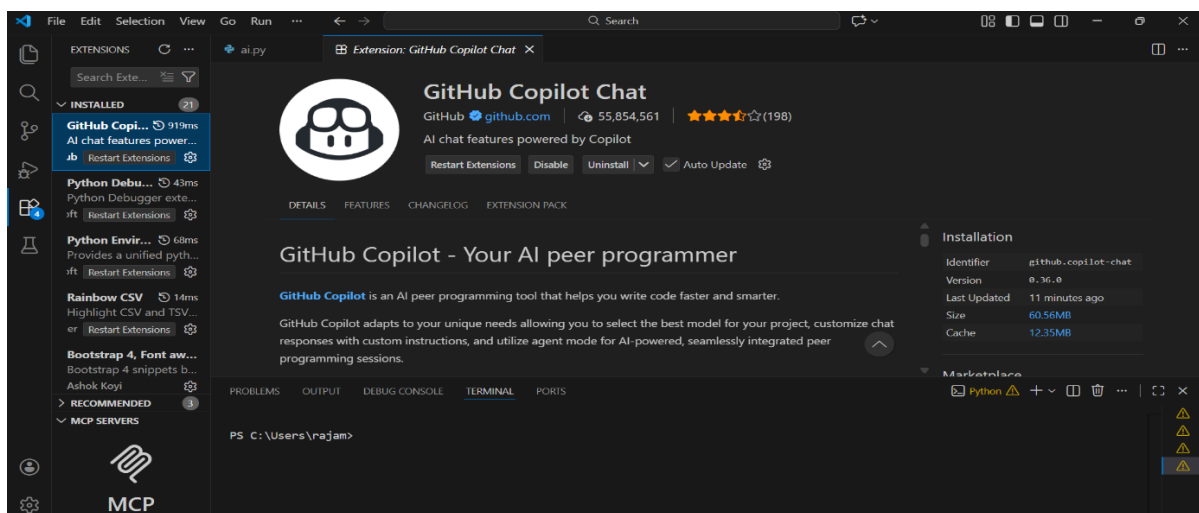
HT.No : 2303A52195

Batch : 35

Environment Setup – *GitHub Copilot and VS Code Integration + Understanding AI-assisted Coding Workflow*

Task 0

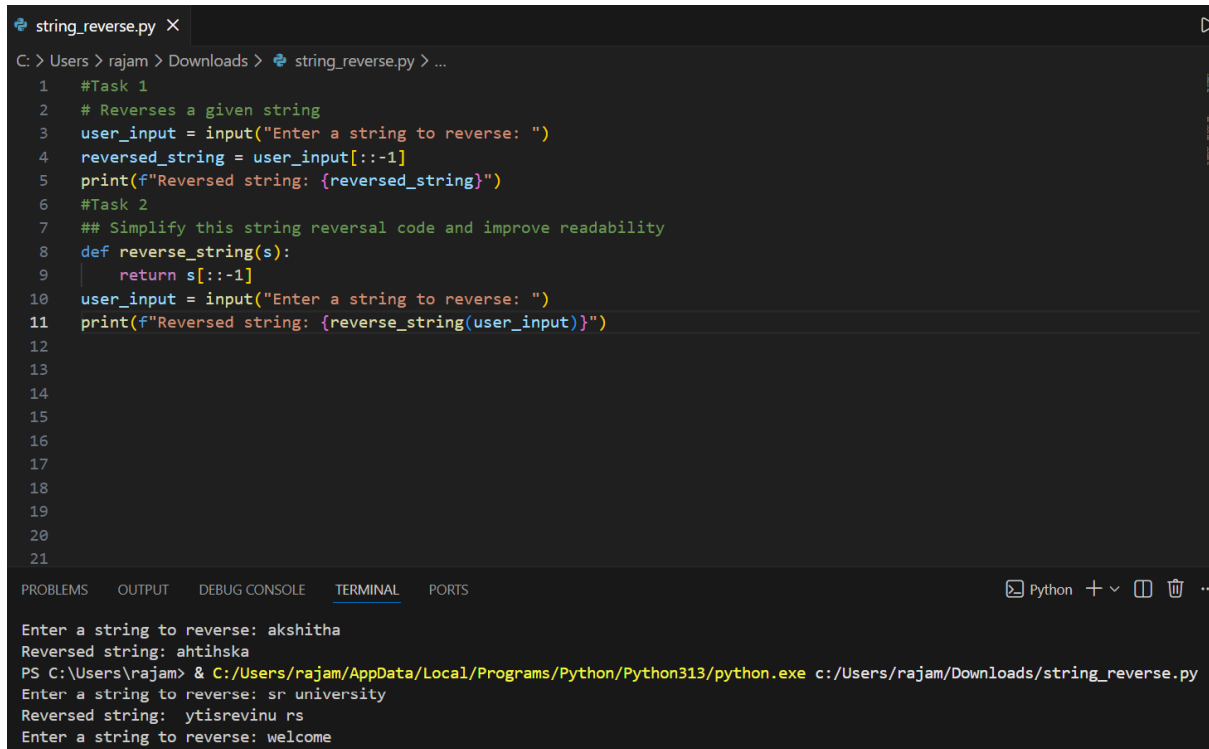
Install and configure GitHub Copilot in VS Code. Take screenshots of each step.



Task 1: AI-Generated Logic Without Modularization (String Reversal Without Functions)

```
string_reverse.py X
C: > Users > rajam > Downloads > string_reverse.py > ...
1  #Task 1
2  # Reverses a given string
3  user_input = input("Enter a string to reverse: ")
4  reversed_string = user_input[::-1]
5  print(f"Reversed string: {reversed_string}")
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
Enter a string to reverse: akshitha
Reversed string: ahtihksa
```

Task 2: Efficiency & Logic Optimization (Readability Improvement)



```
string_reverse.py X
C: > Users > rajam > Downloads > string_reverse.py > ...

1  #Task 1
2  # Reverses a given string
3  user_input = input("Enter a string to reverse: ")
4  reversed_string = user_input[::-1]
5  print(f"Reversed string: {reversed_string}")
6  #Task 2
7  ## Simplify this string reversal code and improve readability
8  def reverse_string(s):
9      return s[::-1]
10 user_input = input("Enter a string to reverse: ")
11 print(f"Reversed string: {reverse_string(user_input)}")
12
13
14
15
16
17
18
19
20
21

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
Python + - [] ...

Enter a string to reverse: akshitha
Reversed string: ahtihksa
PS C:\Users\rajam> & C:/Users/rajam/AppData/Local/Programs/Python/Python313/python.exe c:/Users/rajam/Downloads/string_reverse.py
Enter a string to reverse: sr university
Reversed string: ytisrevinu rs
Enter a string to reverse: welcome
```

Explanation : In Task 2, the original string reversal code was optimized by removing unnecessary variables and simplifying the logic.

The loop and intermediate variable were eliminated, and Python slicing was used directly.

This improved readability and reduced code length.

Both versions have a time complexity of $O(n)$, but the optimized version is more efficient in practice due to fewer operations.

Task 3: Modular Design Using AI Assistance (String Reversal Using Functions)

```
13 #Task 3
14 #Write a Python program using a function to reverse a string.
15 #Add meaningful comments.
16 def reverse_string(s):
17
18     return s[::-1] # Slicing the string to reverse it
19 # Get user input
20 user_input = input("Enter a string to reverse: ")
21 # Call the function and display the reversed string
22 reversed_str = reverse_string(user_input)
23 print(f"Reversed string: {reversed_str}")
24
25
26
27
28
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Enter a string to reverse: sr university
Reversed string: ytisrevinu rs
Enter a string to reverse: welcome
Reversed string: emoclew
Enter a string to reverse: engineering
Reversed string: gnireenigne
```

Explanation

- The function `reverse_string()` encapsulates the string reversal logic
- It uses Python slicing (`[::-1]`) for efficient reversal
- The function returns the reversed string to the caller
- This modular approach allows reuse of the same logic in multiple parts of an application
- Meaningful comments improve code readability and understanding

Task 4: Comparative Analysis – Procedural vs Modular Approach (With vs Without Functions)

Comparison Table

Criteria	Without Functions (Procedural)	With Functions (Modular)
Code Clarity	Logic is mixed with input/output, making it less clear	Logic is separated into a function, improving clarity
Reusability	Code cannot be reused easily	Function can be reused in multiple parts of the application
Debugging Ease	Harder to debug due to lack of separation	Easier to debug and test individual functions
Maintainability	Changes must be made in multiple places	Changes can be made in one function
Scalability	Not suitable for large programs	Suitable for large-scale applications

Task 5: AI-Generated Iterative vs Recursive Fibonacci Approaches (Different Algorithmic Approaches to String Reversal)

➤ A **loop-based** string reversal approach

```

25 #Task 5
26 #Write a Python program to reverse a string using a loop.
27 def reverse_string_loop(s):
28     reversed_str = ""
29     for char in s:
30         reversed_str = char + reversed_str # Prepend each character
31     return reversed_str
32 # Get user input
33 user_input = input("Enter a string to reverse: ")
34 # Call the function and display the reversed string
35 reversed_str = reverse_string_loop(user_input)
36 print(f"Reversed string: {reversed_str}")
37
38

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

Enter a string to reverse: sr university
Reversed string: ytisrevinu rs
Enter a string to reverse: welcome
Reversed string: emoclew
Enter a string to reverse: engineering
Reversed string: gnireenigne

```

➤ A built-in / slicing-based string reversal approach

```

38 #Write a Python program to reverse a string using slicing.
39 def reverse_string_slicing(s):
40     return s[::-1]
41 user_input = input("Enter a string to reverse: ")
42 print(f"Reversed string: {reverse_string_slicing(user_input)}")

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

Enter a string to reverse: sr university
Reversed string: ytisrevinu rs
Enter a string to reverse: welcome
Reversed string: emoclew
Enter a string to reverse: engineering
Reversed string: gnireenigne

```

2. Time Complexity

- **Loop-Based Approach:**
Time complexity is $O(n)$, where n is the length of the string.
- **Built-in Approach:**
Time complexity is also $O(n)$, as the string must be traversed internally.

Conclusion

Although both approaches have the same time complexity, the built-in slicing method is more efficient and readable, making it the preferred choice for practical applications.